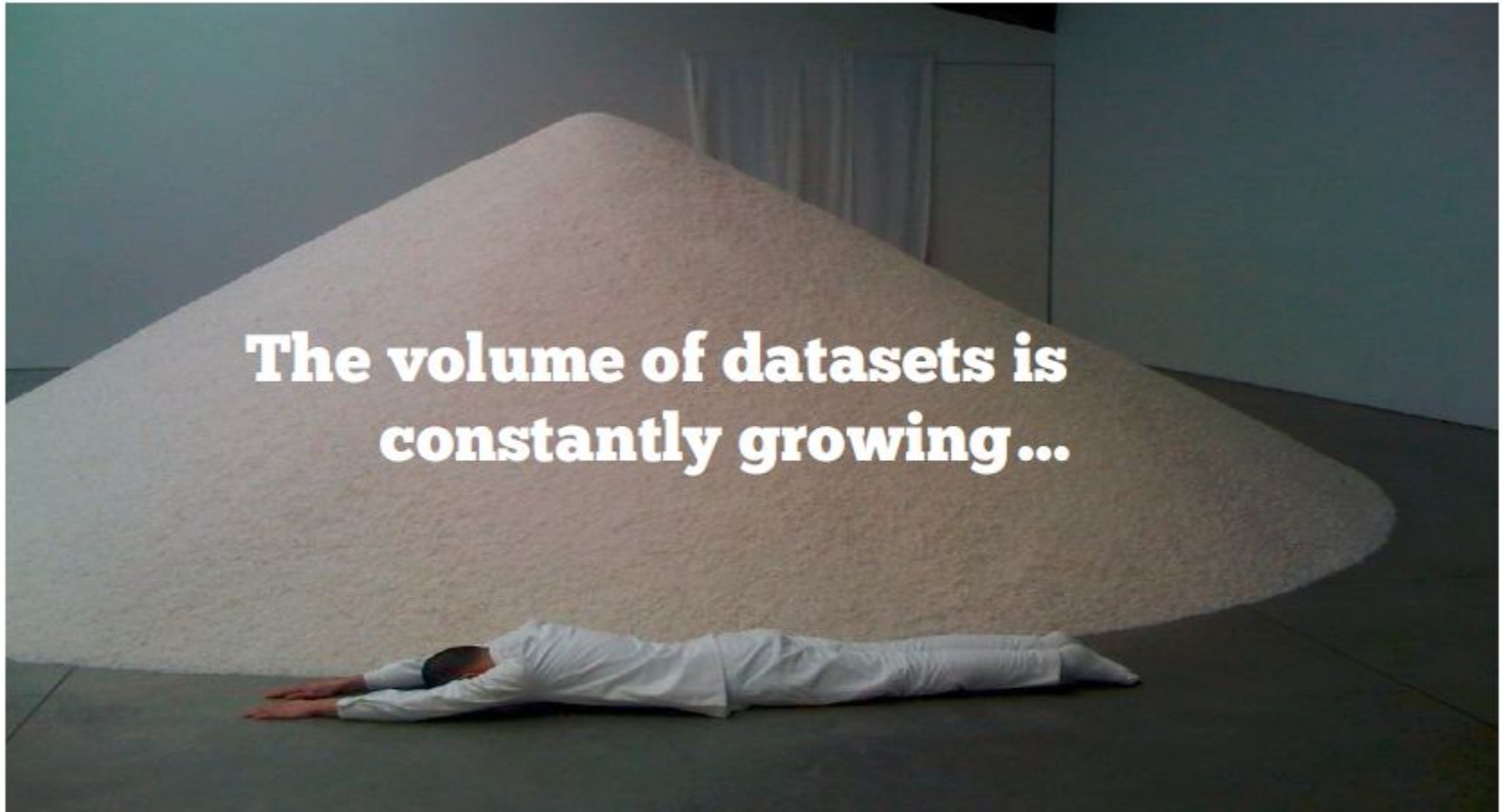
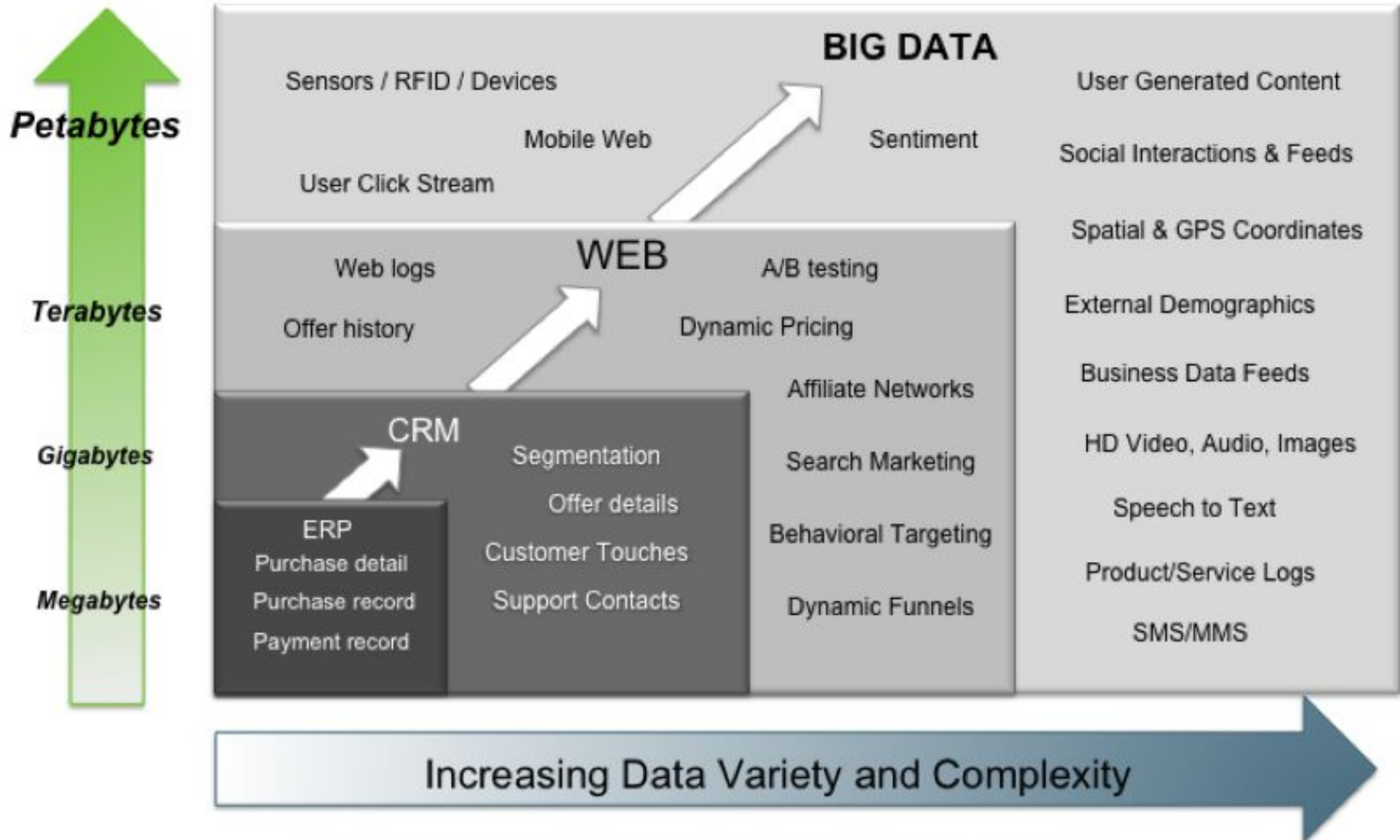


Introduction to Hadoop

Why Big Data ?



Big Data = Transactions $\cap$ Interactions $\cap$ Observations.



Source: Contents of above graphic created in partnership with Teradata, Inc.

**The velocity of data generation is
getting faster and faster...**



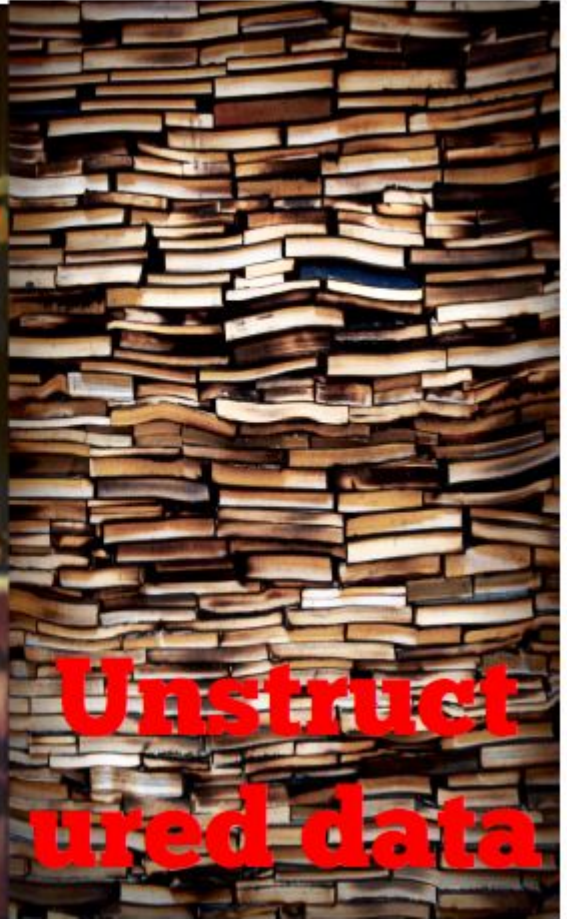
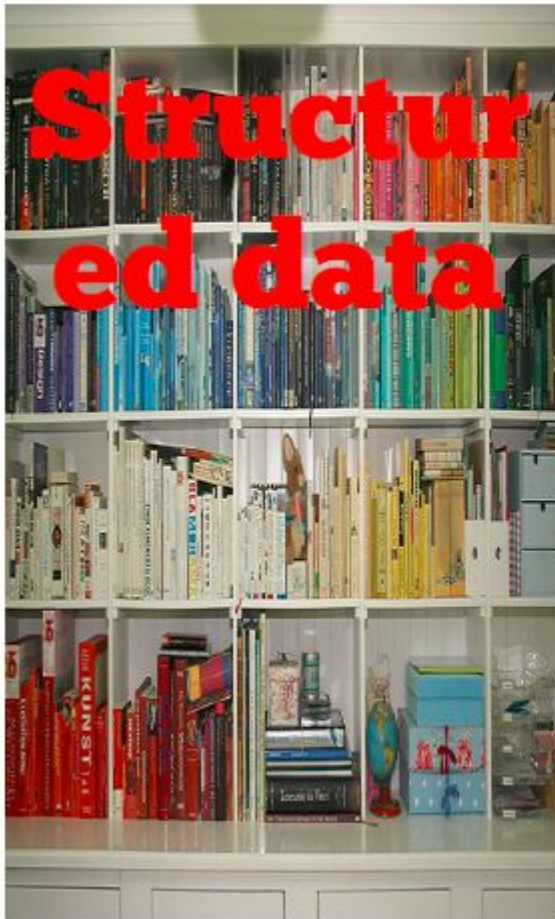
Velocity





The variety of data is increasing...

Variety



3 V's of Big Data



What is Big Data?

- **Data sets** that are **too large** and **complex** to manipulate or interrogate with **standard methods or tools**.

Need

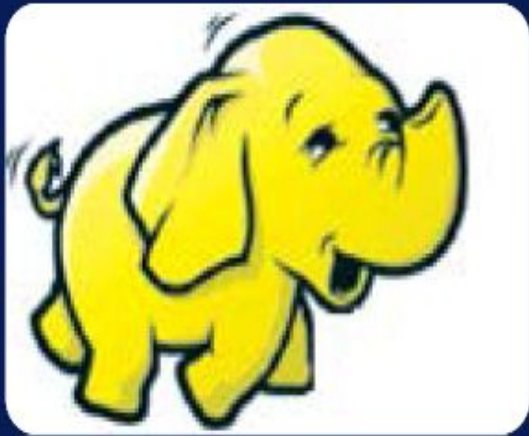
- Technologies basically focus on providing
Real-time,
Scalable,
High performance computing
on this large data sets

What is Hadoop?

Apache Hadoop is an open source software framework for storage and large scale processing of data-sets on clusters of commodity hardware

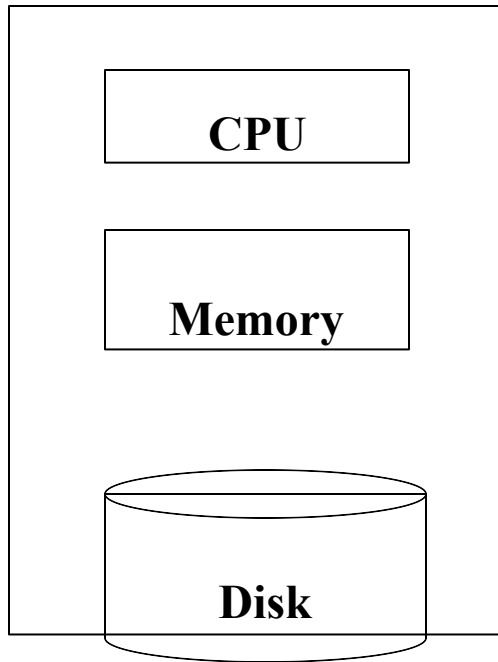
**Hadoop was created by Doug Cutting
and Mike Cafarella in 2005**

**Named the project after son's toy
elephant**



Why Hadoop?

Single Node Architecture



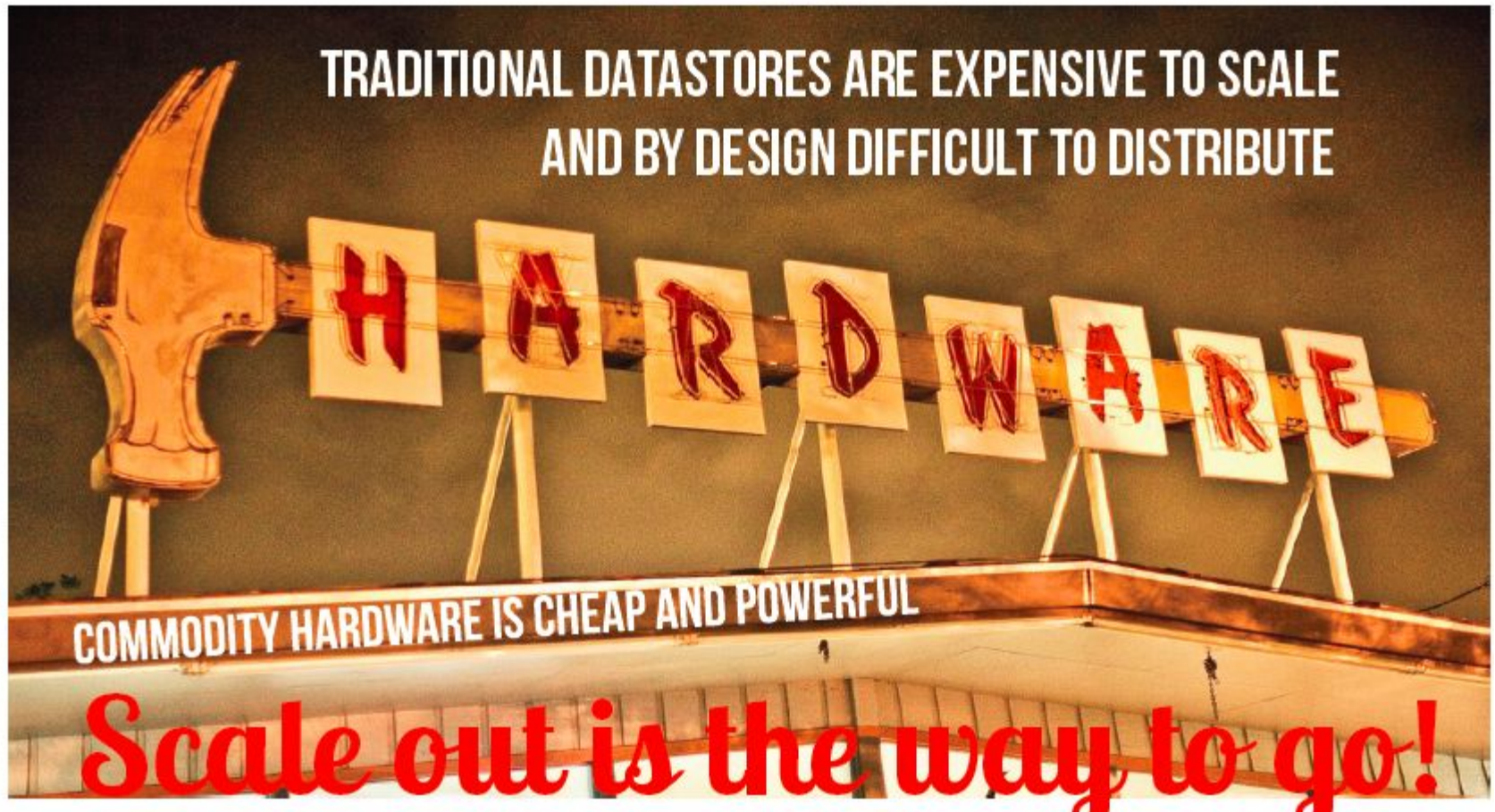
**Machine Learning,
Statistics**

**“Classical” Data
Mining**

Motivation: Google Example

- 10 billion web pages
- Average size of webpage = 20KB
- $10 \text{ billion} * 20\text{KB} = 200 \text{ TB}$
- Disk read bandwidth = 50 MB/sec
- Time to read = 4 million seconds = 46+ days
- Even longer to do something useful with the data

Alternate Solution



Challenges

- Node failures
 - A single server can stay up for 3 years (1000 days)
 - 1000 servers in cluster => 1 failure/day
 - 1M servers in cluster => 1000 failures/day
- How to store data persistently and keep it available if nodes can fail?
- How to deal with node failures during a long-running computation?

Challenges

Network bottleneck

- Network bandwidth = 1 Gbps
- Moving 10TB takes approximately 1 day

Distributed programming is hard!

- Need a simple model that hides most of the complexity

Map-Reduce

- Map-Reduce addresses the challenges of cluster computing
 - Store data redundantly on multiple nodes for persistence and availability
 - Move computation close to data to minimize data movement
 - Simple programming model to hide the complexity of all this magic

Why so popular?

- Hadoop enables computing solution that is
 - Scalable
 - Low Cost
 - Flexible
 - Fault Tolerant

Scalability at Hadoop's core!





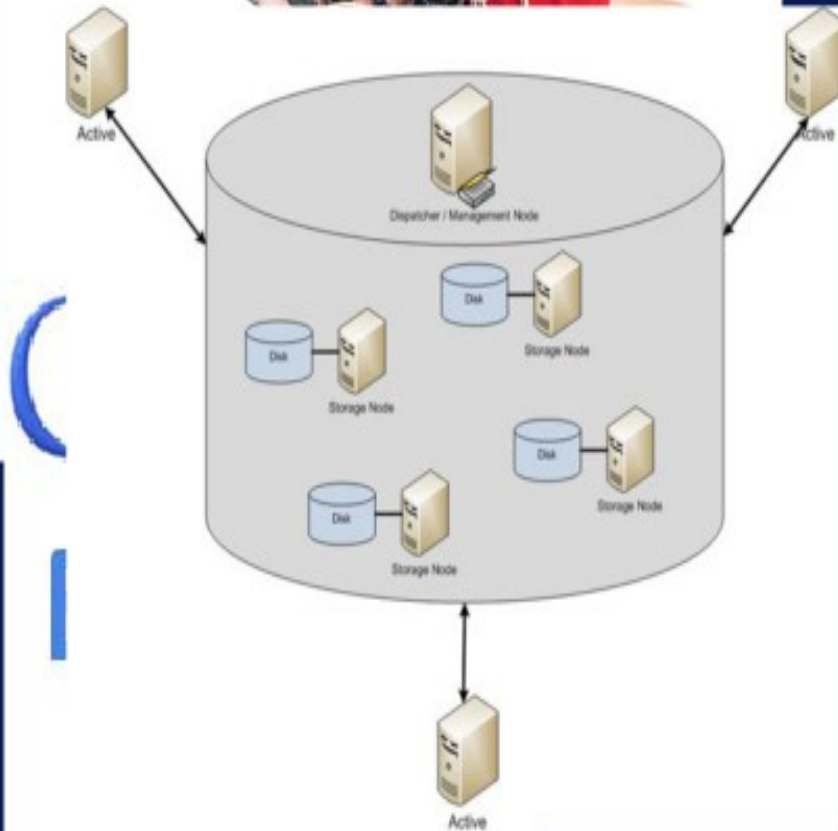
Reliability!
Reliability!
Reliability!



Reliability!
Reliability!
Reliability!



Reliability!
Reliability!
Reliability!



Google File System

New Approach to Data

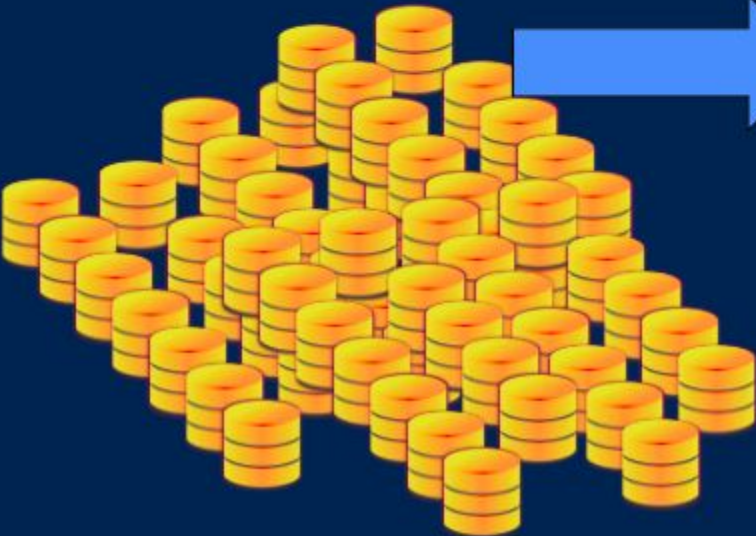
Keep all data





Small Data & Complex Algorithm

Vs.



Large Data & Simple Algorithm

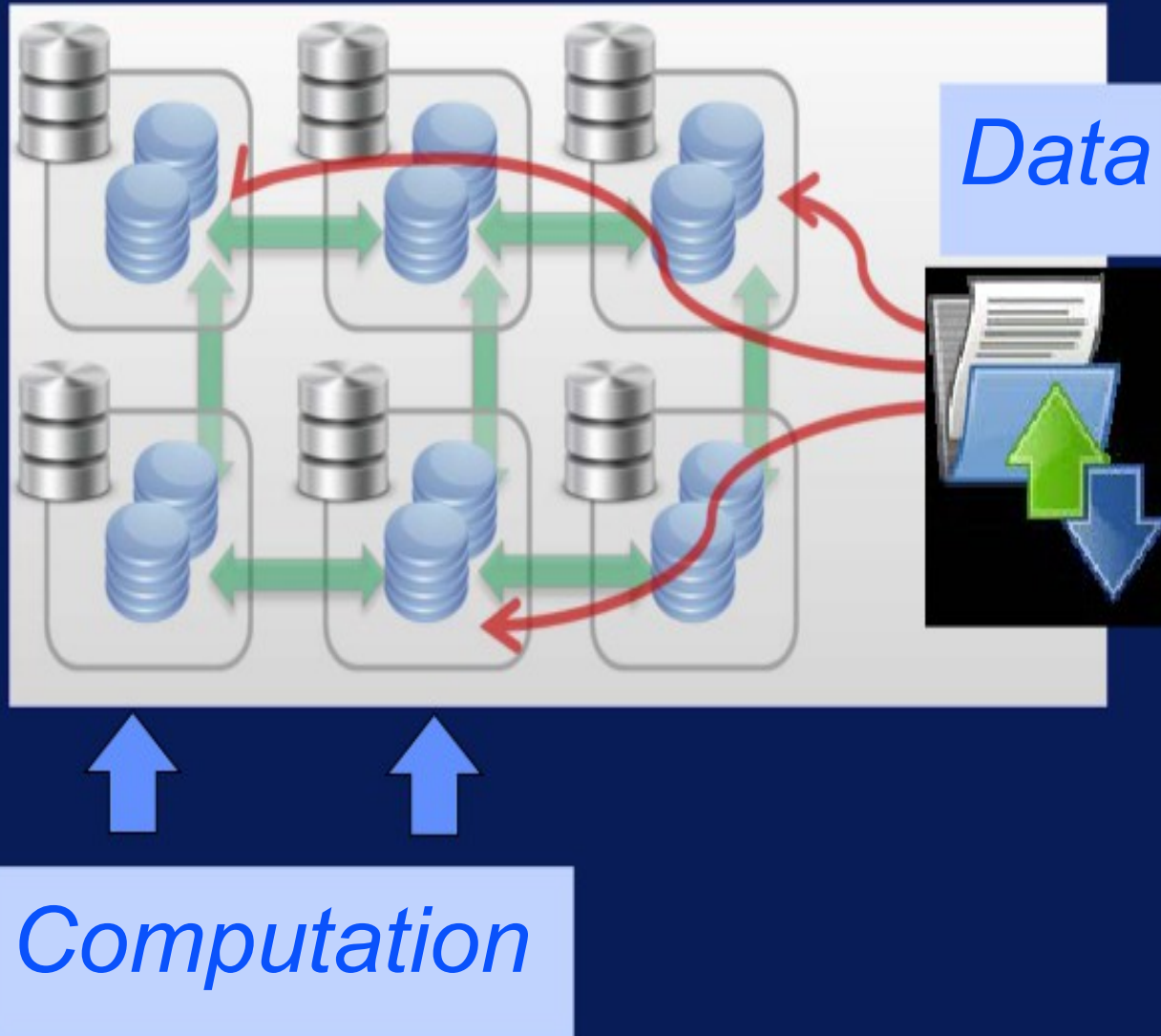
Hadoop

- **Scalable**— **New nodes** can be **added as needed**, and added without needing to change data formats, how data is loaded, how jobs are written, or the applications on top.
- **Low Cost** — Hadoop brings **massively parallel computing** to **commodity servers**. The result is a sizeable decrease in the cost per terabyte of storage, which in turn makes it affordable to model all your data.
- **Flexible**— Hadoop is **schema-less**, and can **absorb any type of data**, structured or not, from any number of sources. Data from multiple sources can be joined and aggregated in arbitrary ways enabling deeper analyses than any one system can provide.
- **Fault tolerant**— When you lose a node, the **system redirects work to another location of the data** and continues processing without missing a fright beat .

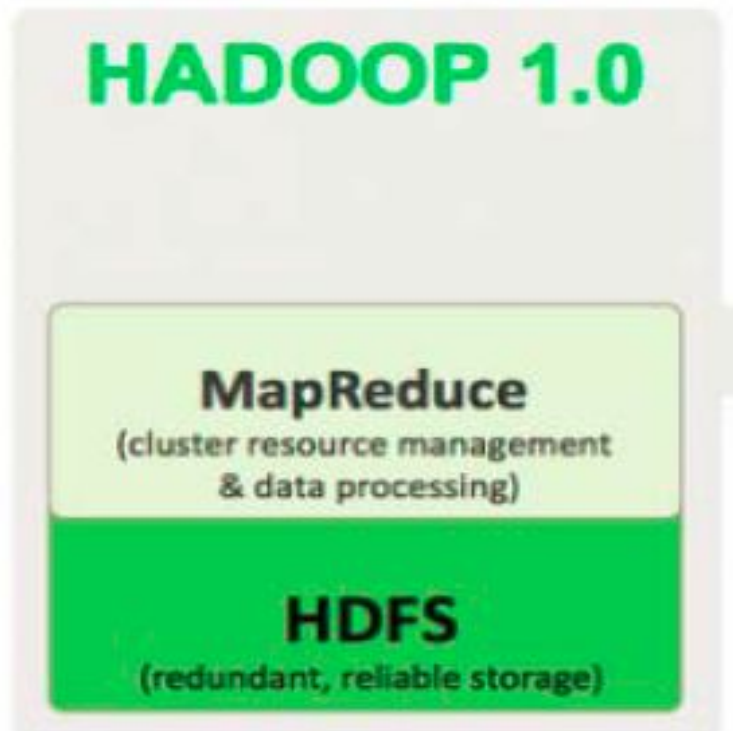
Basic Hadoop Componets

- Hadoop Distributed File System (HDFS)
 - Distributed File System where our data resides
 - Provides built-in redundancy & fault tolerance for all hadoop processing
- Hadoop Map Reduce
 - Programming model for large scale data processing.
 - Brings computation to where data is

Moving Computation to Data



Basic version of hadoop stack



Typical Big-data Problem

- **Iterate over a large number of records**
- **Extract something of interest from each**
- **Shuffle and sort intermediate results**
- **Aggregate intermediate results**
- **Generate final output**



Hadoop Assumptions

Hadoop written with large clusters of computer in mind.

- Hardware *will* fail.
- Processing will be run in batches.; High throughput low latency.
- Applications run on HDFS have large data sets.
- High aggregate data bandwidth and scale to hundreds of nodes in a single cluster.
- Applications need a **write-once-read-many** access model..
- Portability is important.

Apache Framework

Hadoop Modules

Apache Framework

Basic Modules

Hadoop Common

**Hadoop Distributed File System
(HDFS)**

Hadoop

YARN

Hadoop MapReduce

Hadoop Common

- Contains libraries and utilities needed by other hadoop modules.

Apache Framework Basic Modules

Hadoop Common

Hadoop Distributed File System
(HDFS)

Hadoop

YARN
Hadoop MapReduce

Hadoop Distributed File System

- Distributed File System that stores data on commodity machine, providing very high aggregate bandwidth.

Apache Framework Basic Modules

Hadoop Common

Hadoop Distributed File System
(HDFS)

Hadoop

YARN

Hadoop MapReduce

Hadoop YARN

- Resource Management platform responsible for managing computing resources in clusters & using them for scheduling user application

Apache Framework Basic Modules

Hadoop Common

Hadoop Distributed File System
(HDFS)

Hadoop

YARN
Hadoop MapReduce

Hadoop MapReduce

- Programming model for large scale data processing.

Hadoop Common Package

- Contains libraries and utilities to run hadoop modules.

Hadoop Distributed File System (HDFS)

Distributed, Scalable, and portable system
written in Java for the Hadoop framework

HDFS

- Refer HDFS PDF file

MapReduce Framework

- *Software framework – for writing parallel data processing applications*
- *MapReduce job splits data into chunks*
- *Map tasks process data chunks*
- *Framework sorts map output*
- *Reduce tasks use sorted map data as input*

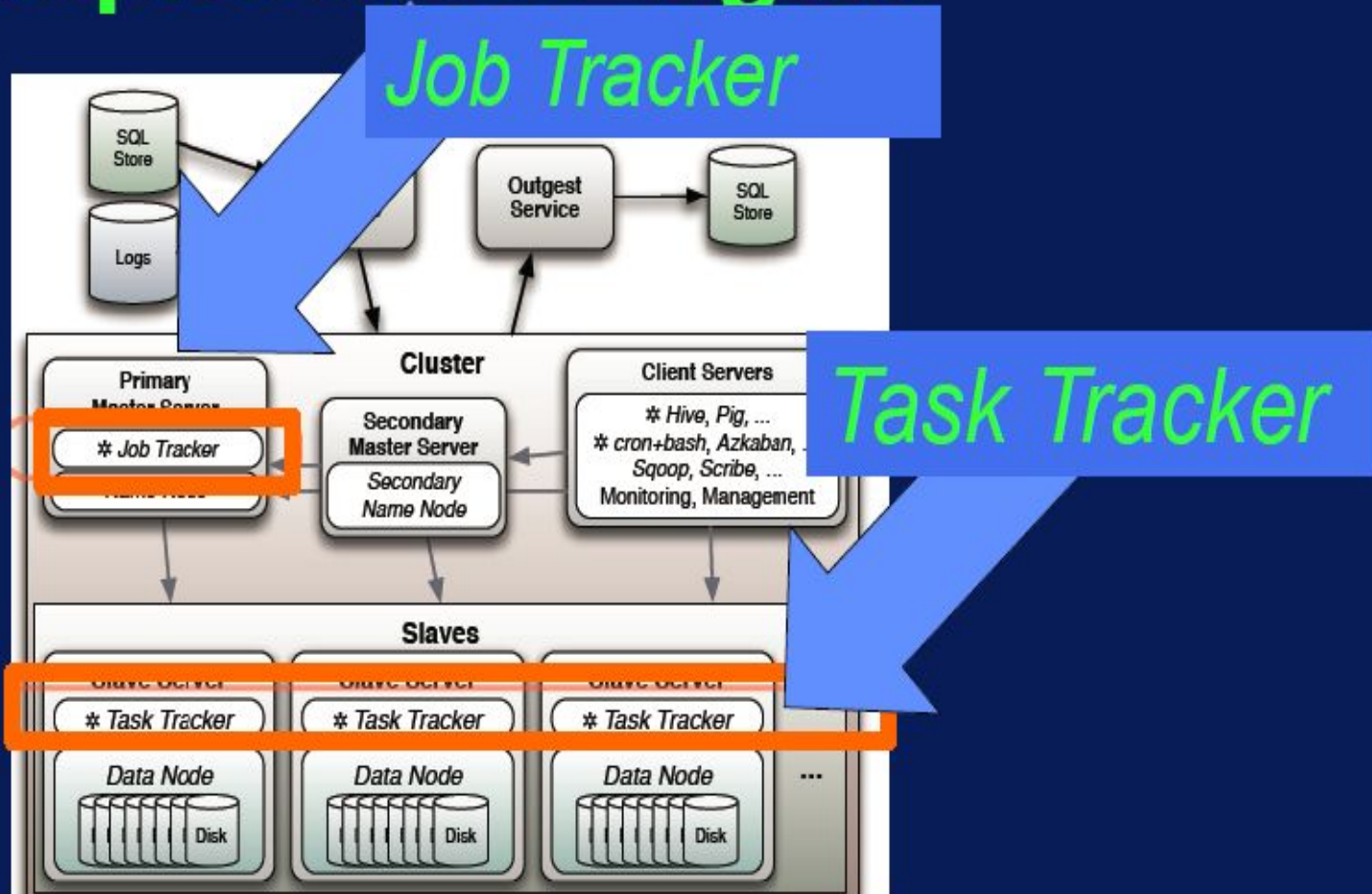
MapReduce Framework

- *Typically compute and storage nodes are the same.*
- *MapReduce tasks and HDFS running on the same nodes*
- *Can schedule tasks on nodes with data already present.*

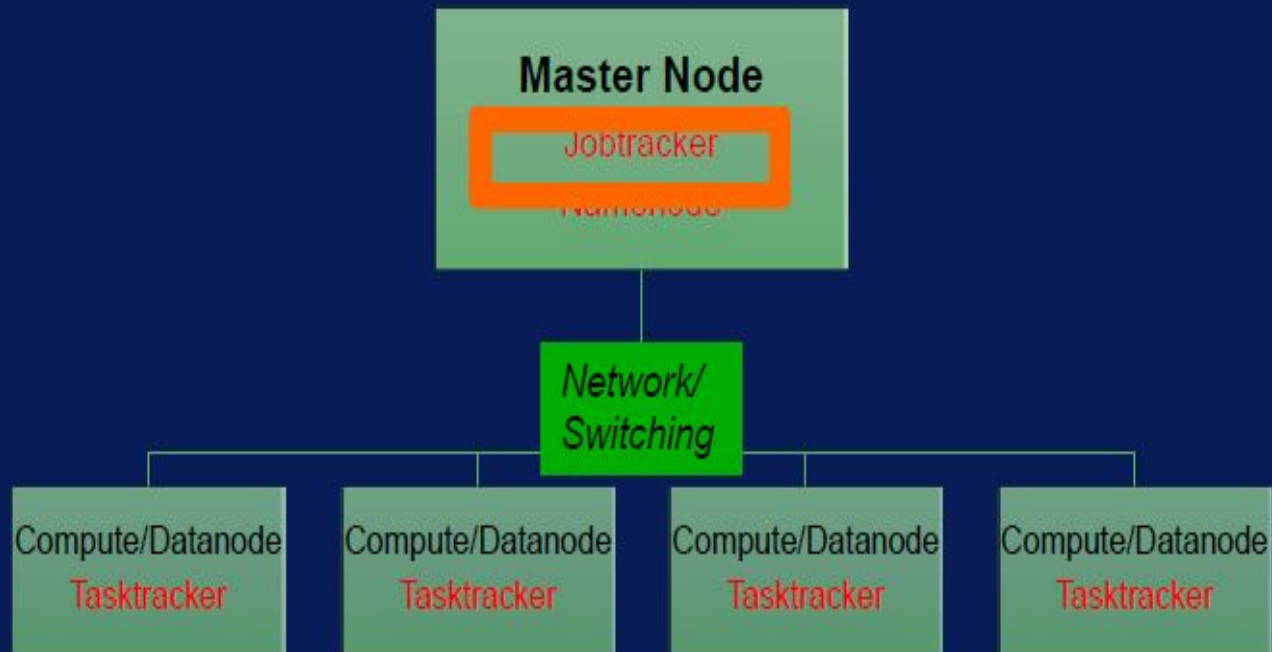
Original MapReduce Framework

- *Single master JobTracker*
- *JobTracker schedules, monitors, and re-executes failed tasks.*
- *One slave TaskTracker per cluster node*
- *TaskTracker executes tasks per JobTracker requests.*

MapReduce Engine



Original Hadoop Architecture



Summary of main Components of Map Reduce

- **Job tracker**- manages jobs and resources in the cluster. Schedules each map on the Task tracker running on same data node as the underlying block.
- **Task Tracker** - Task Tracker are slaves which are deployed on each machine in the cluster. Responsible for running map and reduce tasks assigned by Job Tracker.
- **JobHistoryServer**- daemon that saves historical information about completed tasks/applications.

YARN: NexGen MapReduce

- *Main idea – Separate resource management and job scheduling/monitoring.*
- *Global ResourceManager (RM)*
- *NodeManager on each node*
- *ApplicationMaster – one for each application*

Hive
Query

PIG
Script

HCatalog
Metadata Services

MapReduce
Distributed Processing

YARN
Resource Scheduling and Negotiation

HDFS
Distributed Storage

Other YARN
Frameworks

HBase
Non-relational Database

Other Projects
Ambari, Avro, Cassandra, Oozie,
Zookeeper, etc.

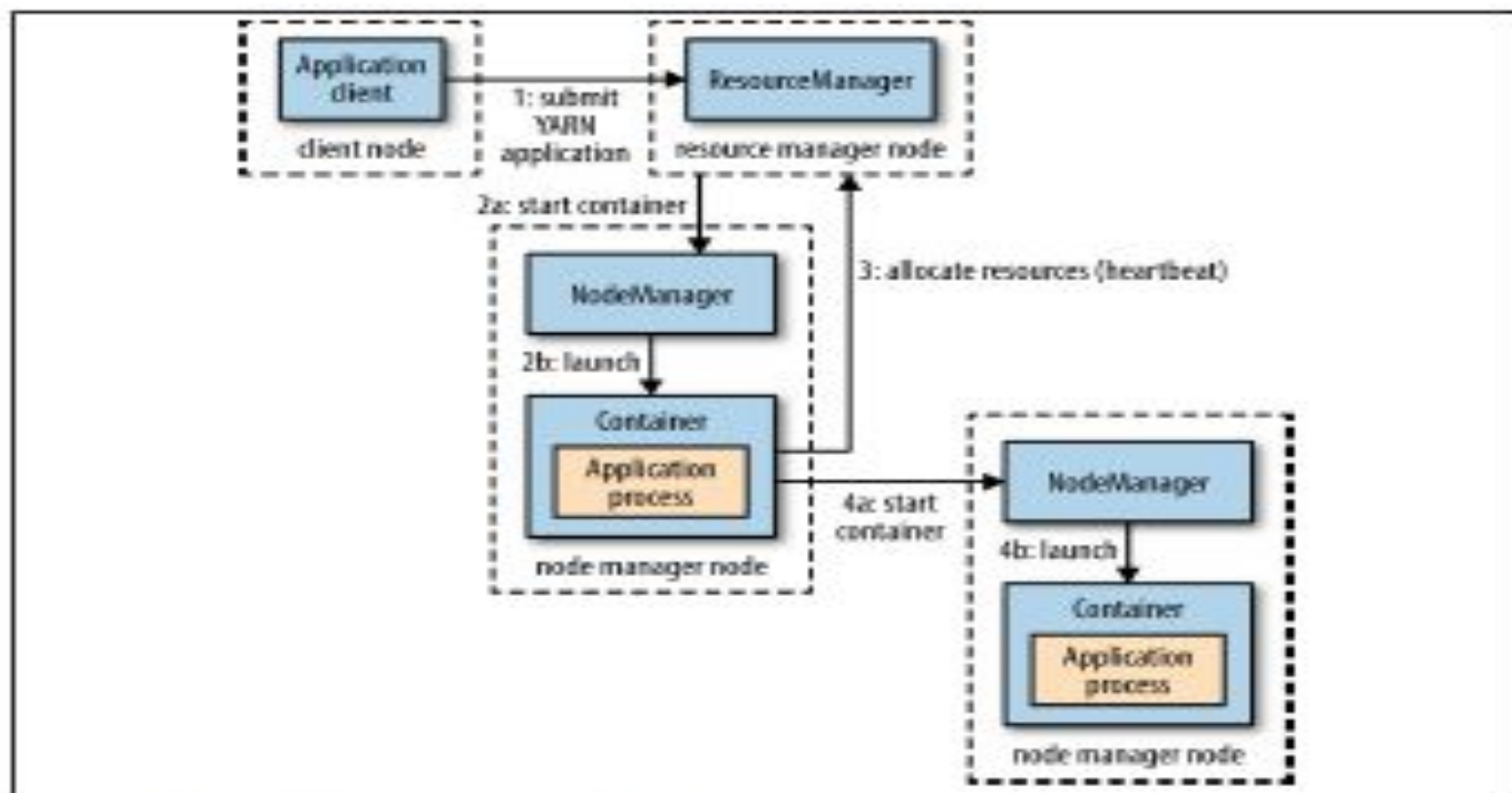


Figure 4-2. How YARN runs an application

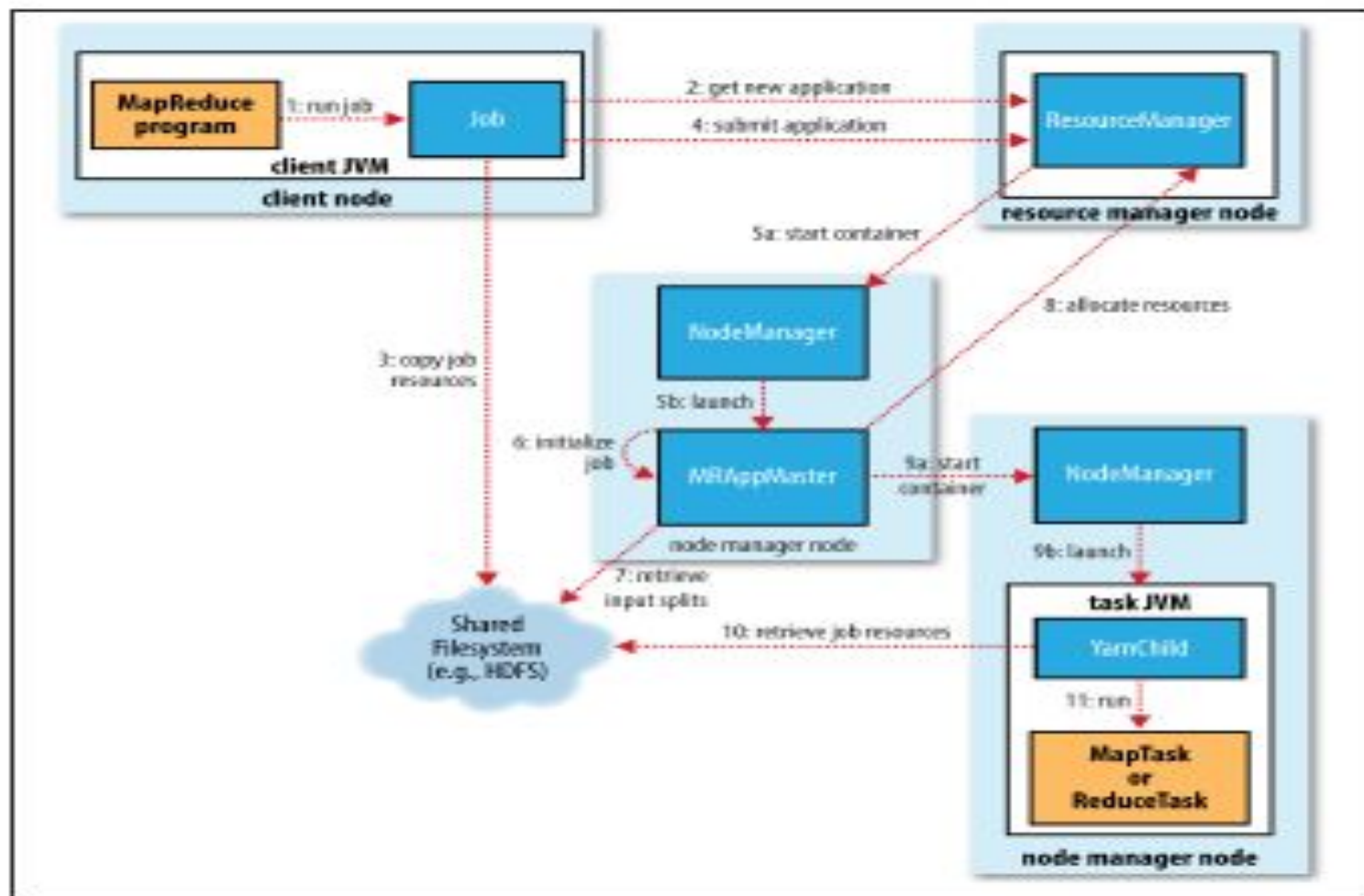


Figure 7-1. How Hadoop runs a MapReduce job



Apache Hadoop Ecosystem



Ambari

Provisioning, Managing and Monitoring Hadoop Clusters



Scoop
Data Exchange



Zookeeper
Coordination



Oozie
Workflow



Pig
Scripting



Mahout
Machine Learning

R Connectors
Statistics



Hive
SQL Query



Hbase
Columnar Store



YARN Map Reduce v2

Distributed Processing Framework

HDFS

Hadoop Distributed File System



Flume
Log Collector

Apache Sqoop

- Tool designed for efficiently transferring bulk data between Apache Hadoop and structured datastores such as relational databases



Sqoop

- SQL-to-Hadoop
- Transfers data in both ways bwn relational systems and HDFS or other data store like Hive and Hbase
- Easy parallel database import/export

HBase

- Distributed, nonrelational (column) database
- Sits on the top of HDFS
- Can Store extremely large data.
- Based on Google BigTable.
- Random ,real-time, read-write access to Big data.

Pig

- High level data flow language (Pig Latin) on Map Reduce
- Easy to understand
- Originally developed by Yahoo.
- Inbuilt operators and functions to transform input to desired output.
- It is developed at Yahoo!

Pig Example

```
users = LOAD 'users.txt' USING PigStorage(',') AS (name,  
    age);  
pages = LOAD 'pages.txt' USING PigStorage(',') AS (user,  
    url);  
  
filteredUsers = FILTER users BY age >= 18 and age <=50;  
joinResult = JOIN filteredUsers BY name, pages by user;  
grouped = GROUP joinResult BY url;  
summed = FOREACH grouped GENERATE group,  
    COUNT(joinResult) as clicks;  
sorted = ORDER summed BY clicks desc;  
top10 = LIMIT sorted 10;  
  
STORE top10 INTO 'top10sites';
```

Hive

- DataWarehousing layer on the top of Hadoop
- Allows analysis and query using large data sets in HDFS using SQL like language (Hive-QL)
- Developed at Facebook
- Hive Query Language
 - Basic-SQL : Select, From, Join, Group-By
 - Equi-Join, Muti-Table Insert, Multi-Group-By
 - Batch query

**Eg; SELECT * FROM purchases WHERE price > 100
GROUP BY storeid**

Oozie

- Workflow scheduler system to manage apache hadoop jobs
- Coordinator for jobs
- Supports – MapReduce, Apache HIVE, PIG, SQOOP,

Mahout

- Machine-learning tool
- Distributed and scalable machine learning algorithms on the Hadoop platform
- Building intelligent applications easier and faster

Mahout Use Cases

- Yahoo: Spam Detection
- Foursquare: Recommendations
- SpeedDate.com: Recommendations
- Adobe: User Targetting
- Amazon: Personalization Platform

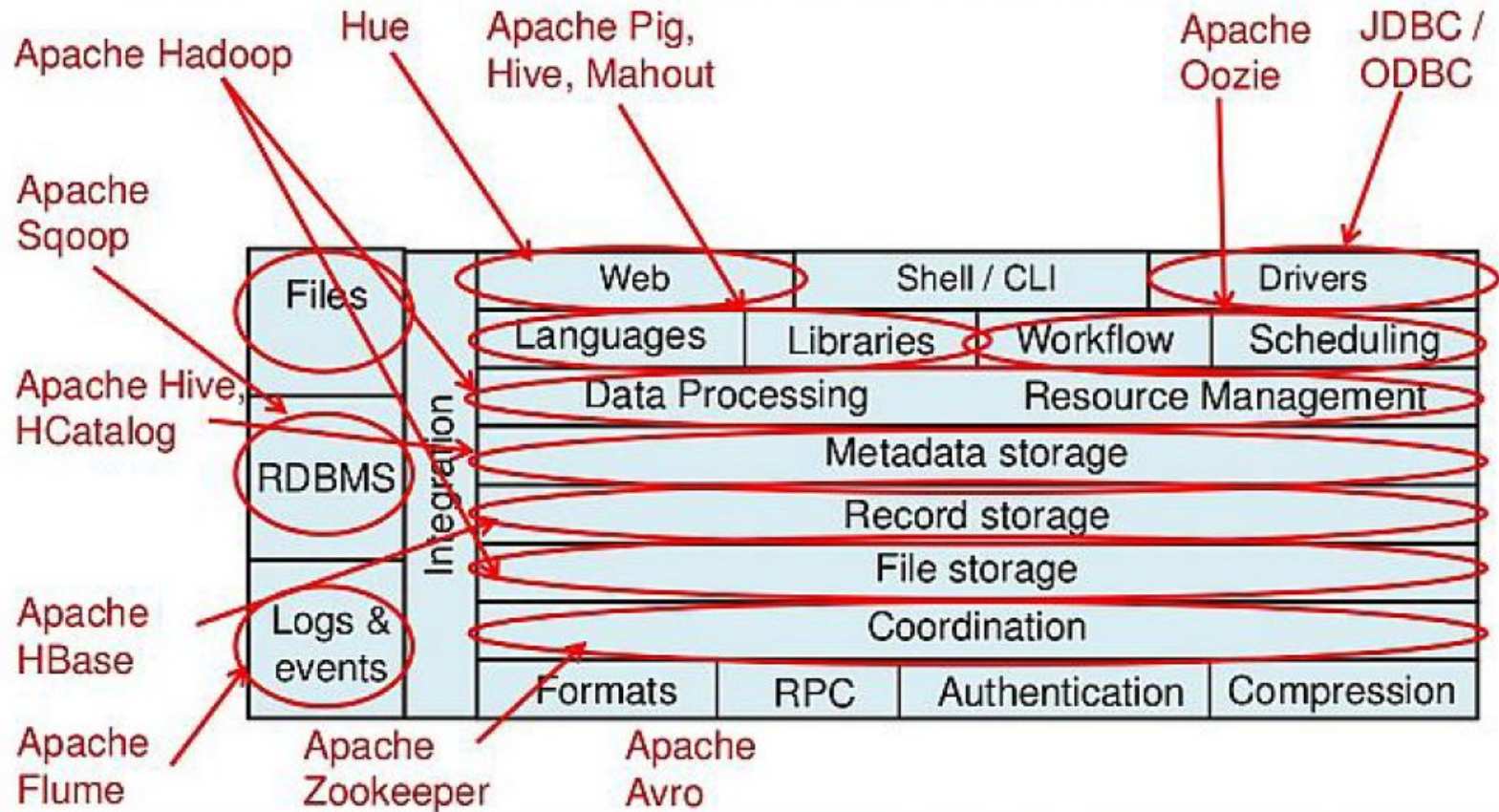
Zookeeper

- A centralized service for maintaining
 - Configuration information
 - Providing distributed synchronization
- A set of tools to build distributed applications that can safely handle partial failures
- ZooKeeper was designed to store coordination data
 - Status information
 - Configuration
 - Location information

The wider Hadoop Ecosystem

- Ambari, Zookeeper (managing & monitoring)
- HBase, Cassandra (database)
- Hive, Pig (data warehouse and query language)
- Mahout (machine learning)
- Chukwa, Avro, Oozie, Giraph, and many more

Wider hadoop Ecosystem



Who is using Hadoop?

- Facebook – Hadoop, Hive, Hbase for datawarehousing and real-time application serving.
- Yahoo- Hadoop for data analytics, machine learning, search ranking, email antispam, ad optimization, ETL
- Twitter, eBay, Samsung, Rackspace, JP Morgan, LinkedIn, AOL etc.
- Google – Caffeine, It version of MapReduce to create web index for storing crawl data

Hadoop Limitations

- Security
 - Security feature is disable due to sheer complexity
 - Doesn't not provide encryption at storage and network level.
- Vulnerable
 - Hadoop framework almost written in Java
- Not Fit for small data
 - Inefficient in handling small files.
 - Doesn't support random access to data.

Hadoop Limitation

- Potential stability issues
 - Hadoop is a open source frame work.
- General Limitation
 - Google mentions – Hadoop is not only answer for big data.
 - Google use Cloud data flow as a solution.